

Холимог Микрофронтэнд архитектур: React ба JSF ашигласан кейс судалгаа

(Hybrid Microfrontend Architecture: A React and JSF Case Study)

Гүйцэтгэсэн: Ц.Биндэрцэцэг (22B1NUM0027)

Удирдагч: Доктор (Ph.D.), Dipl.-Ing., Dipl.-Inf. Б.Батням



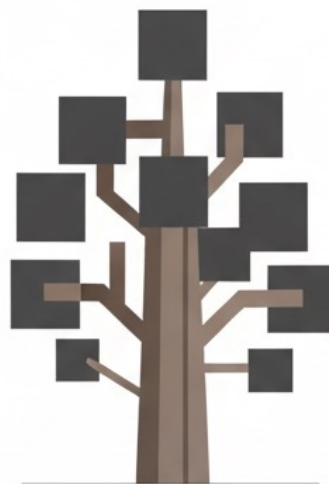
МУИС - Мэдээллийн Технологи, Электроникийн Сургууль

- 1 Удиртгал
- 2 Зорилго, Зорилт
- 3 Судалгааны арга зүй
- 4 Туршилт, Үр дүн
- 5 Дүгнэлт

Шаардлага: Хуучин системийг шинэчлэх.

Big Bang:

- **Давуу тал:** Хуучин болон шинэ системийг нэгэн зэрэг ажиллуулах шаардлагагүй.
- **Сул тал:** Бүх зүйлийг нэг дор өөрчилдөг тул алдаа гарах магадлал маш өндөр.

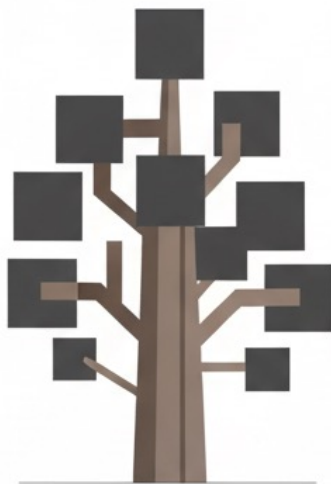


Хуучин систем

Strangler Fig:

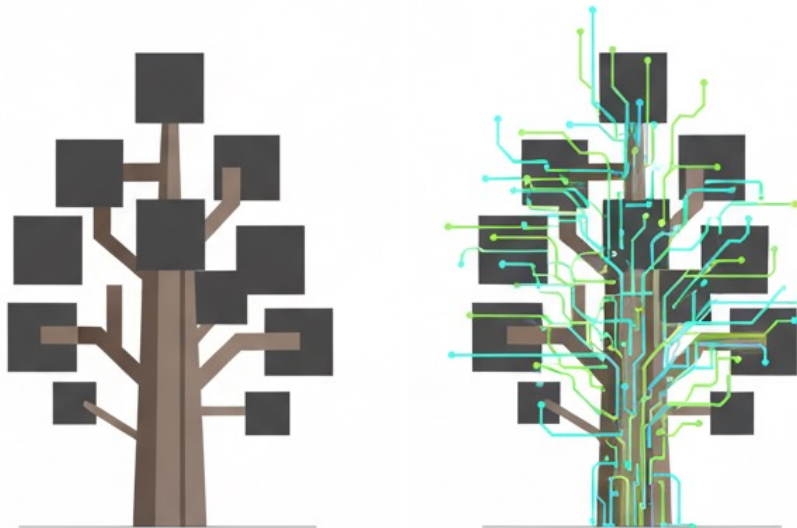
- **Давуу тал:** Системийг унтраахгүйгээр шинэчлэлтүүдийг аажим аажмаар хийгдэг.
- **Сул тал:** Хуучин болон шинэ системийг хоёуланг нь зэрэг ажиллуулах шаардлагатай.

Шийдэл: “Strangler Fig” үлгэр загвар [7]



Identify

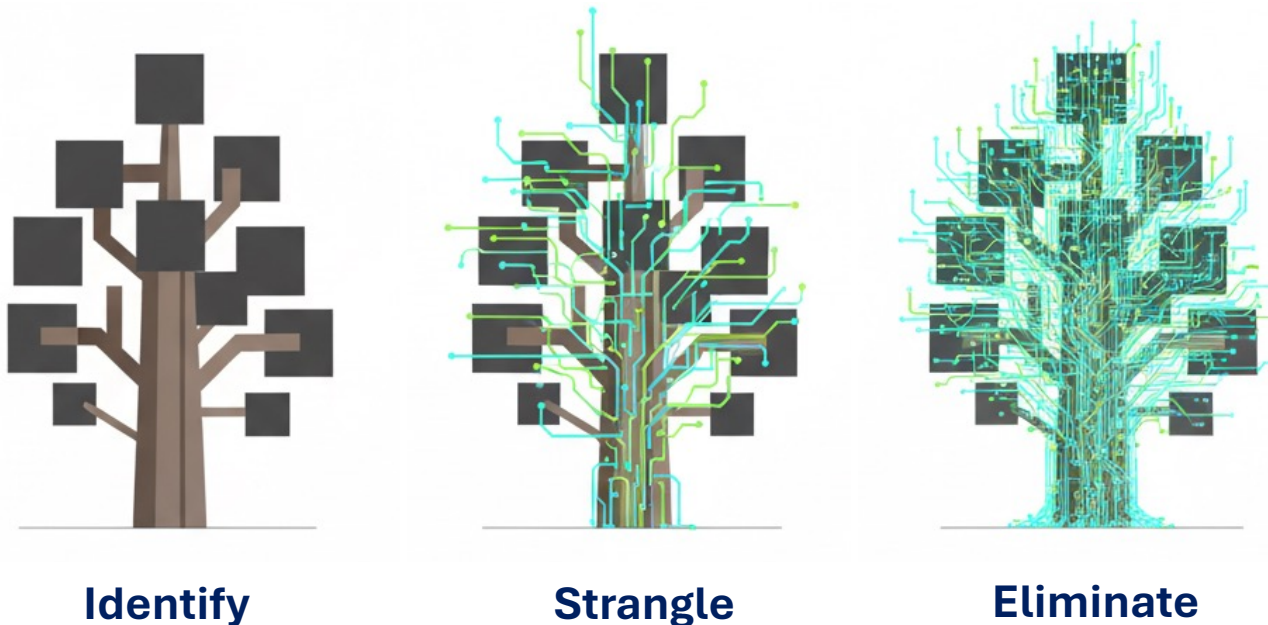
Шийдэл: “Strangler Fig” үлгэр загвар [7]



Identify

Strangle

Шийдэл: “Strangler Fig” үлгэр загвар [7]

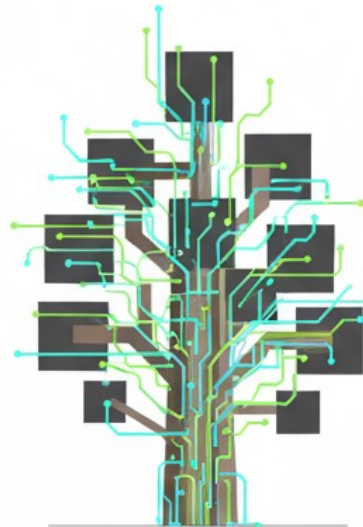


Шийдэл: “Strangler Fig” үлгэр загвар [7]

"Холимог архитектур"-ын
хэрэгцээ



Identify



Strangle



Eliminate

- 1 Удиртгал
- 2 Зорилго, Зорилт**
- 3 Судалгааны арга зүй
- 4 Туршилт, Үр дүн
- 5 Дүгнэлт

Зорилго:

МУИС-ийн Дипломын ажлын удирдлагын системийн хэрэглэгчийн интерфэйсд React болон JSF технологийг хослуулсан микрофронтэнд архитектурыг туршиж, ISO/IEC 25010-ын дагуу чанарын үзүүлэлтүүдээр үнэлэн үр ашгийг тодорхойлох

Зорилт:

1. **Судалгаа хийх** зорилтоор микрофронтэнд архитектурын онол, арга зүйг судлах;
2. **Туршилтын загвар боловсруулах** зорилтоор React, JSF ашиглан холимог архитектур бүхий системийн туршилтын загварыг хөгжүүлэх;
3. **Туршилт ба үнэлгээ хийх** зорилтоор боловсруулсан загвар дээр туршилт хийж, ISO/IEC 25010 стандарт болон DORA, ATAM аргачлалаар үнэлж, дүгнэнэ.

- 1 Удиртгал
- 2 Зорилго, Зорилт
- 3 Судалгааны арга зүй**
- 4 Туршилт, Үр дүн
- 5 Дүгнэлт

Судалгааны арга зүй



1. Судалгааны хүрээ тодорхойлох:

"Scopus", "DiVA", ISO/IEC 25010 ПХ-ийн чанарын үзүүлэлтүүд сонгох [1].

Сэдвийн судалгаа

Өнөөгийн ололт амжилт

Техникийн баримт бичгүүд

ISO/IEC 25010

Үнэлгээний арга зүй

2. Туршилтын загвар (PoC) боловсруулах:

Холимог микрофронтэнд архитектурын суурь загвар

Туршилтын загвар

Демо программ

Туршилт

Холимог микрофронтэнд

Санал болгож буй шийдлийг гаргах

Өгөгдөл цуглуулах

Туршилтийн үр дүн

ISO/IEC 25010

Санал болгож буй шийдлийг хэмжих

5. Өгөгдөл шинжлэх:

ATAM аргачлалаар архитектурын шийдвэрийн "Trade-off" болон "мэдрэмтгий" цэгийг шинжлэх

Өгөгдөл шинжлэх

ATAM

Үр дүнг танилцуулах

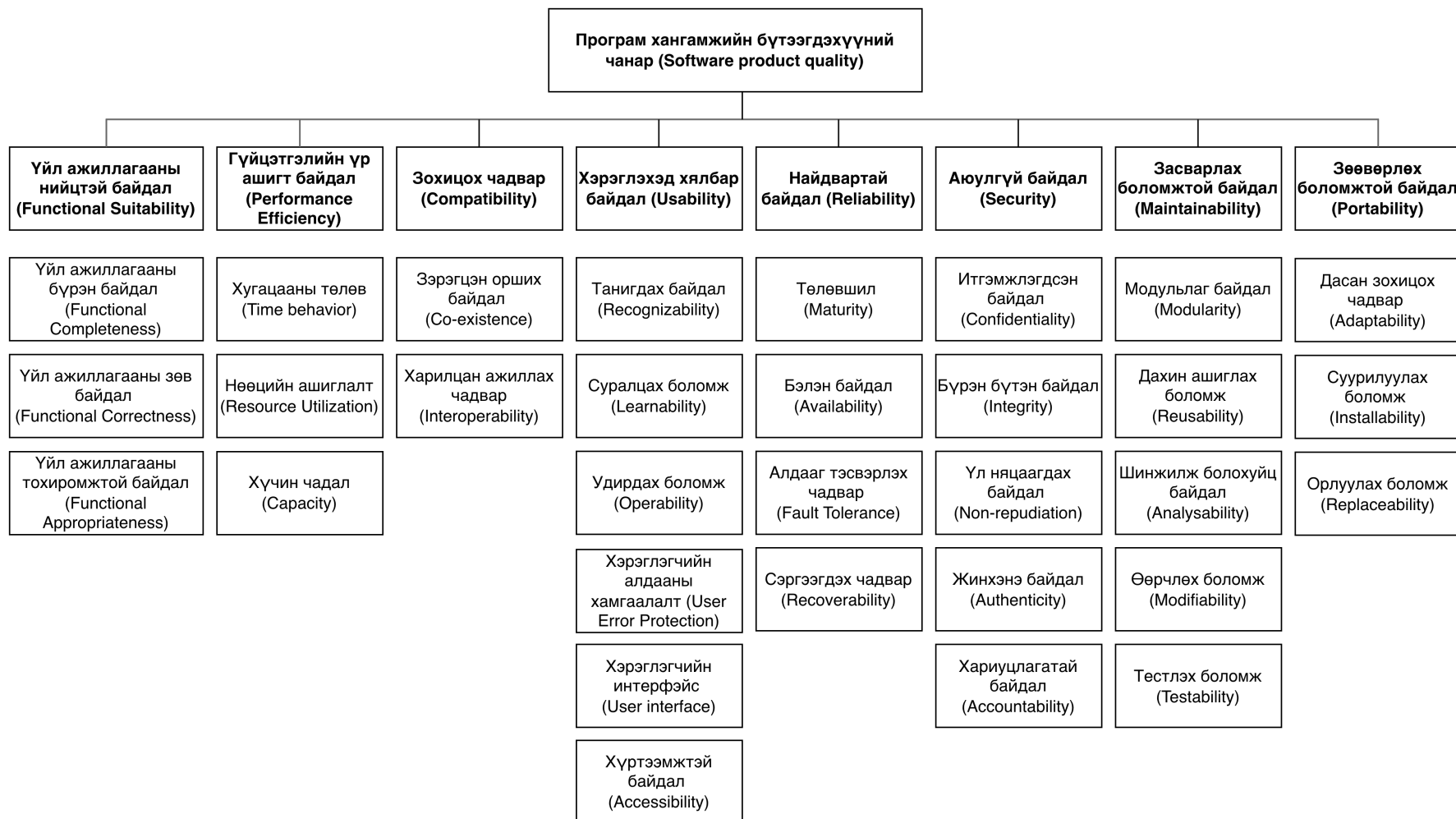
4. Кэйс судалгаа:

JSF болон MFE + WebFlux архитектурыг ижил орчинд, 4 бүлэг хэмжүүрээр харьцуулан хэмжих

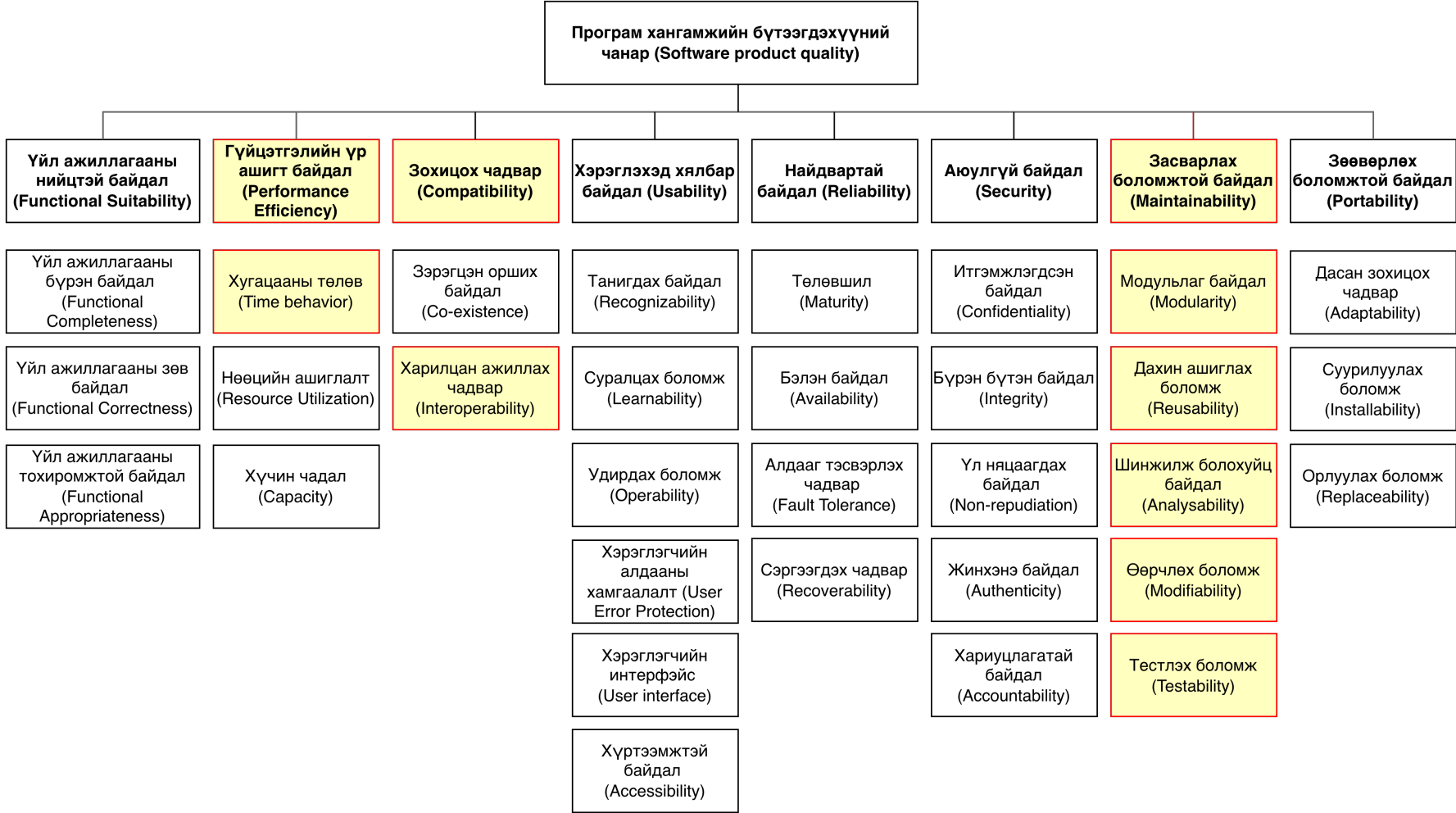
3. Туршилт:

Микрофронтэнд шийдлүүдийг бодит хөгжүүлэлтээр хэрэгжүүлж, чанарын зарчмуудтай нийцүүлэн давтамжтайгаар үнэлэх.

ISO/IEC 25010: ПХ-ийн чанарын үзүүлэлт



ISO/IEC 25010: Сонгосон чанарын үзүүлэлт



Гүйцэтгэлийн үр
ашигт байдал
(Performance
Efficiency)

ПХ-ийн чанарын үзүүлэлтийн тайлбар

Хугацааны төлөв
(Time behavior)

1. Хугацааны төлөв (Time behavior)

- хариу үйлдэл үзүүлэх хугацаа, боловсруулалтын хурд, нэвтрүүлэх чадвар [1]

Нөөцийн ашиглалт
(Resource Utilization)

Хүчин чадал
(Capacity)

Зохицох чадвар
(Compatibility)

ПХ-ийн чанарын үзүүлэлтийн тайлбар

Зэрэгцэн орших
байдал
(Co-existence)

Харилцан ажиллах
чадвар
(Interoperability)

2. Харилцан ажиллах чадвар (Interoperability)

- хоорондоо мэдээлэл солилцож, уг солилцсон мэдээллээ ашиглах [1]

Засварлах
боломжтой байдал
(Maintainability)

ПХ-ийн чанарын үзүүлэлтийн тайлбар

Модульлаг байдал
(Modularity)

3. Модульлаг байдал (Modularity)

- өөрчлөлт оруулахад бусад хэсгүүддээ хамгийн багаар нөлөө үзүүлэх

Дахин ашиглах
боломж
(Reusability)

4. Дахин ашиглах боломж (Reusability)

- нэгээс олон системд, эсвэл өөр шинэ нөөц бүтээхэд ашиглагдах

Шинжилж болохуйц
байдал
(Analysability)

5. Шинжилж болохуйц байдал (Analysability)

- өөрчлөлт оруулахад гарах нөлөөллийг үнэлэх,
- доголдол болон алдааны шалтгааныг оношлох,
- өөрчлөх шаардлагатай хэсгүүдийг тодорхойлох процесс хэр үр дүнтэй, үр ашигтай байх

Өөрчлөх боломж
(Modifiability)

6. Өөрчлөх боломж (Modifiability)

- алдаа доголдол үүсгэхгүйгээр, эсвэл одоо байгаа чанарыг нь бууруулахгүйгээр үр дүнтэй, үр ашигтай өөрчлөлт оруулах

Тестлэх боломж
(Testability)

7. Тестлэх боломж (Testability)

- тестийн шалгуурыг тогтоож, уг шалгуурыг хангасан эсэхийг баталгаажуулах тестийг гүйцэтгэх [1]

Цул архитектур

- Нэг баг бүх давхаргыг хариуцна,
- Бүх давхарга нь нэг кодын санд бичигдэнэ.

Төслийн цар хүрээ нэмэгдэхийн хэрээр:

- модулиуд улам бүр хамааралтай болно,
- Нарийн төвөгтэй байдал нэмэгдэнэ [2].



Цул архитектур: Хэвтээ хуваалт

Давуу тал:

- Энгийн бүтэц, түргэн эхлүүлэх,
- Нэг орчинд дебаг, тест хийх [3].

Сул тал:

- Засвар үйлчилгээ хийх хэцүү,
- Өргөтгөх, өөрчлөх боломж хязгаарлагдмал,
- Ижил технологи, фреймворк ашиглах шаардлагатай [2].



Микросервис архитектур

Давуу тал:

- Бие даасан нэвтрүүлэлт,
- Алдаа тусгаарлалт,
- Багийн зэрэгцээ хөгжүүлэлт [3].

Сул тал:

- CI/CD, DevOps ур чадварын шаардлага өснө,
- Сүлжээний хамаарал, логжилт төвөгтэй [4].



Нэг хуудаст аппликэйшн (SPA)

Хуудсыг бүтнээр нь дахин ачаалахгүйгээр, зөвхөн шаардлагатай хэсгийг динамикаар шинэчилнэ.

Систем болон кодын сан томрох тусам

- засвар үйлчилгээ хийх, удирдах үйл явц маш хүндрэлтэй болдог.
- Нэмэлт хөгжүүлэлт хийхийн тулд бүхэлд нь дахин бичих өндөр эрсдэлтэй [5].



Микрофронтэнд архитектур

Давуу тал:

- UI-г хэсэгчлэн шинэчилнэ,
- Баг тус бүр модуль дээр бие даан хөгжүүлж, байршуулна.

Сул тал:

- Интеграцийн тохиргоо, стандарт шаардана,
- Гүйцэтгэл, dependency зөрчилдөх эрсдэлтэй [6].



Файлын бүтэц:

Үндсэн аппликэйшн

src/main/webapp	📁
index.xhtml	
students.xhtml	
topics.xhtml	
resources/host.css	
WEB-INF	📁
target	📁
pom.xml	

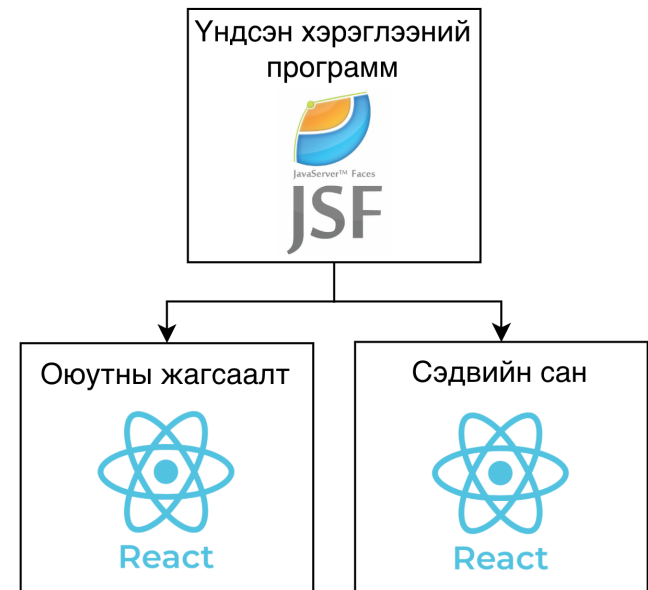
Оюутны жагсаалт

src	📁
index.js	
bootstrap.js	
App.jsx	
App.module.css	
package.json	
package-lock.json	
webpack.config.js	
babel.config.js	

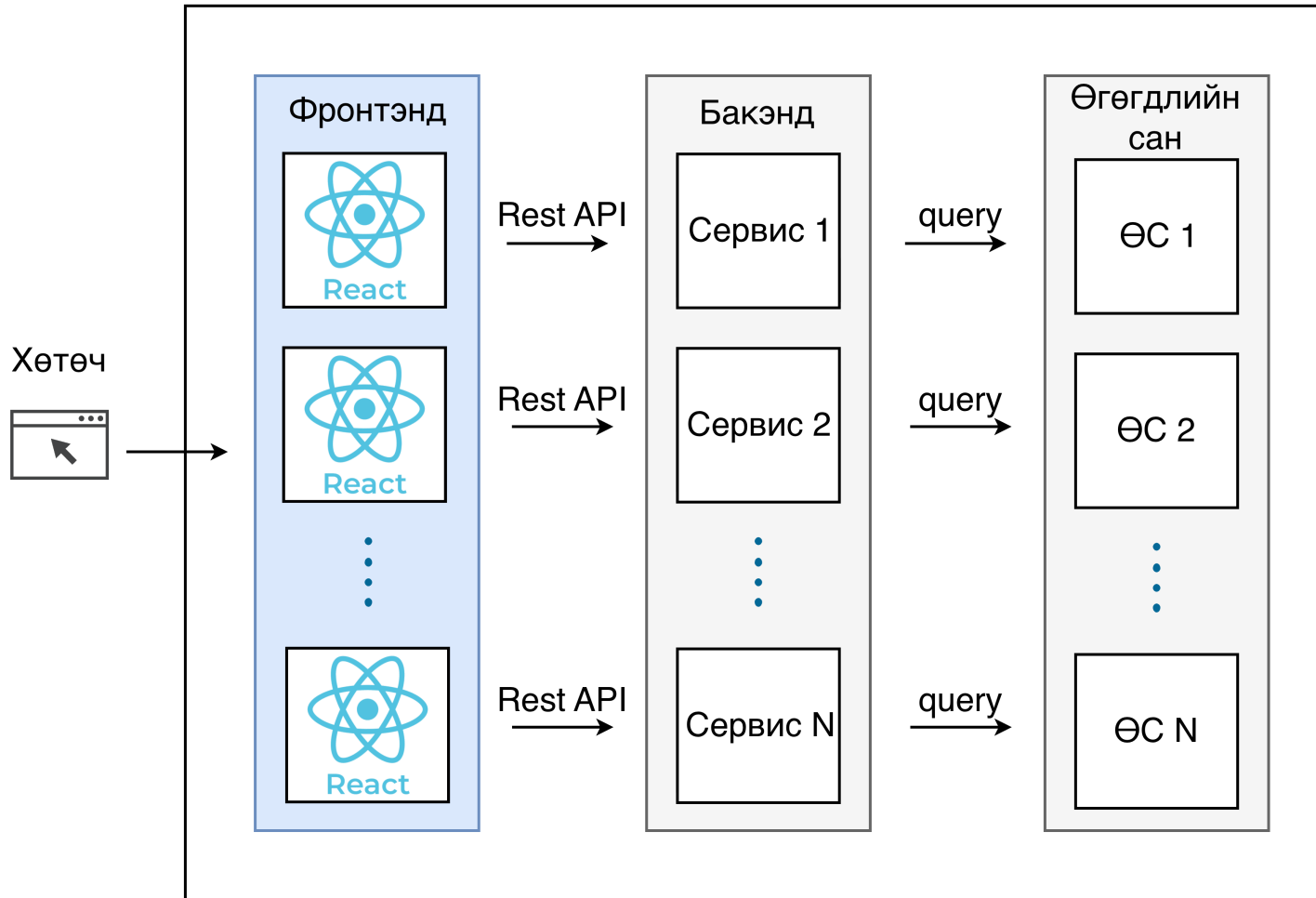
Сэдвийн сан

src	📁
index.js	
bootstrap.js	
App.jsx	
App.module.css	
package.json	
package-lock.json	
webpack.config.js	
babel.config.js	

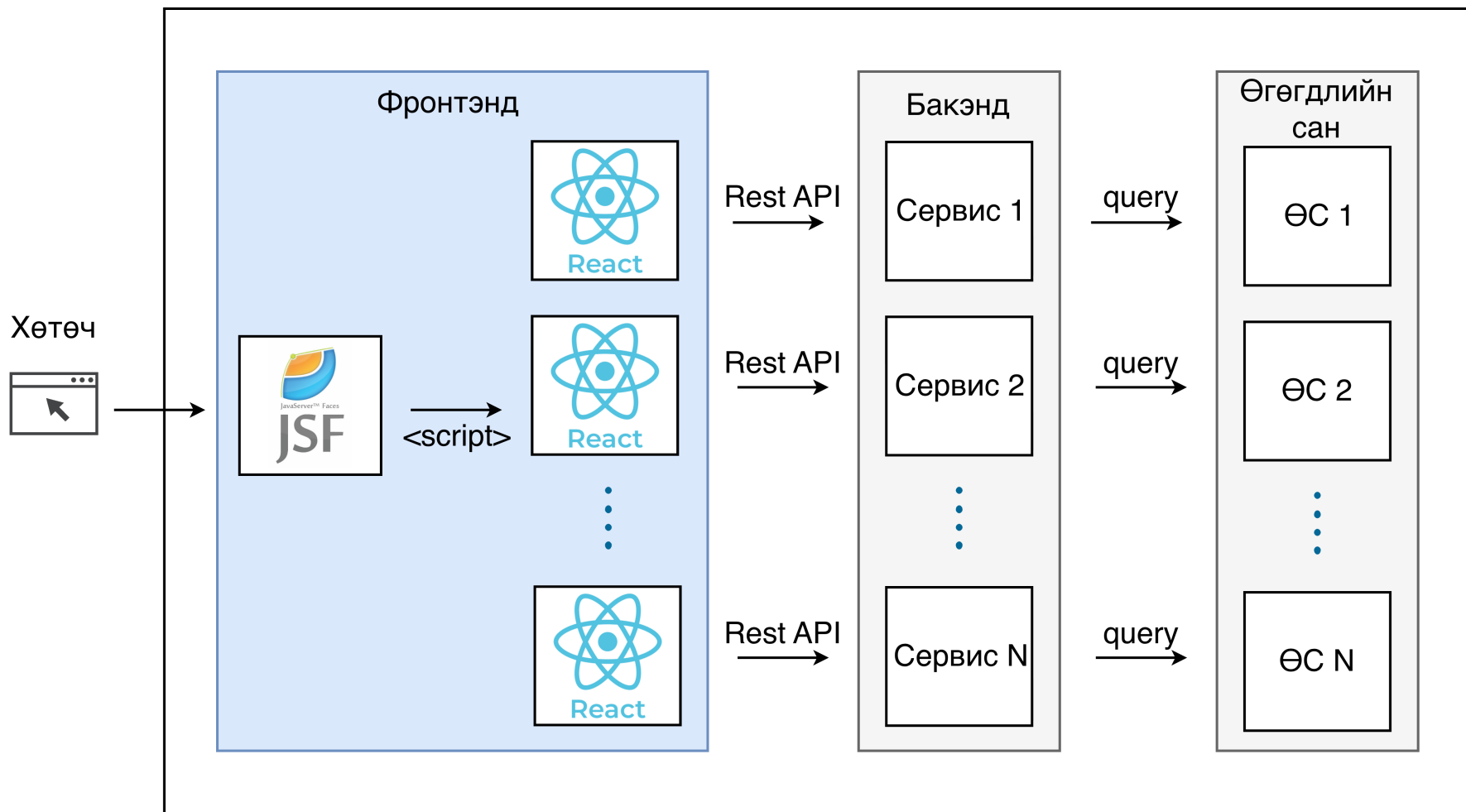
Бүтэц:



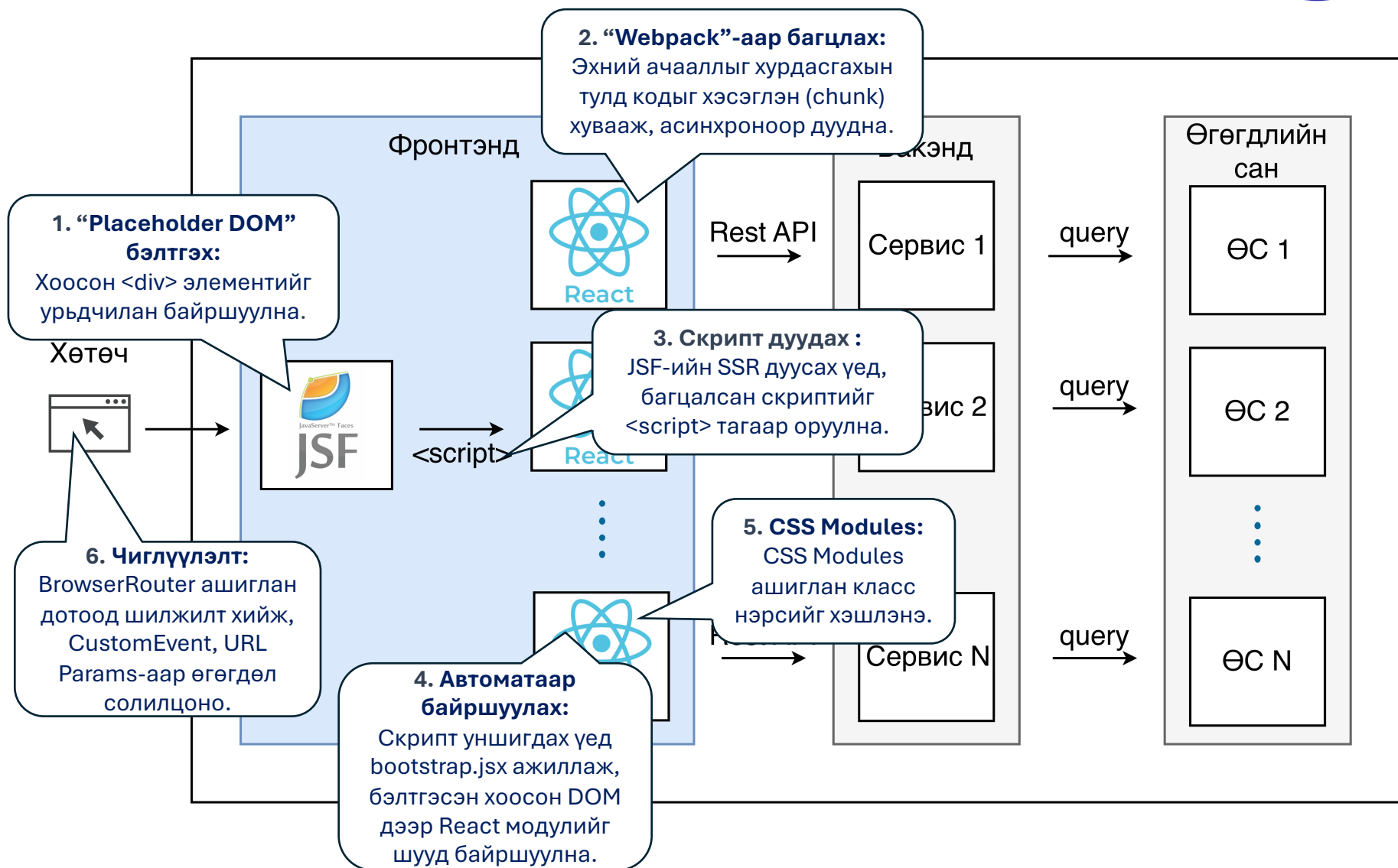
Одоо байгаа микрофронтэнд шийдэл



Санал болгож буй холимог микрофронтэнд шийдэл



Ажиллах үеийн интеграци



- 1 Удиртгал
- 2 Зорилго, Зорилт
- 3 Судалгааны арга зүй
- 4 Туршилт, Үр дүн**
- 5 Дүгнэлт

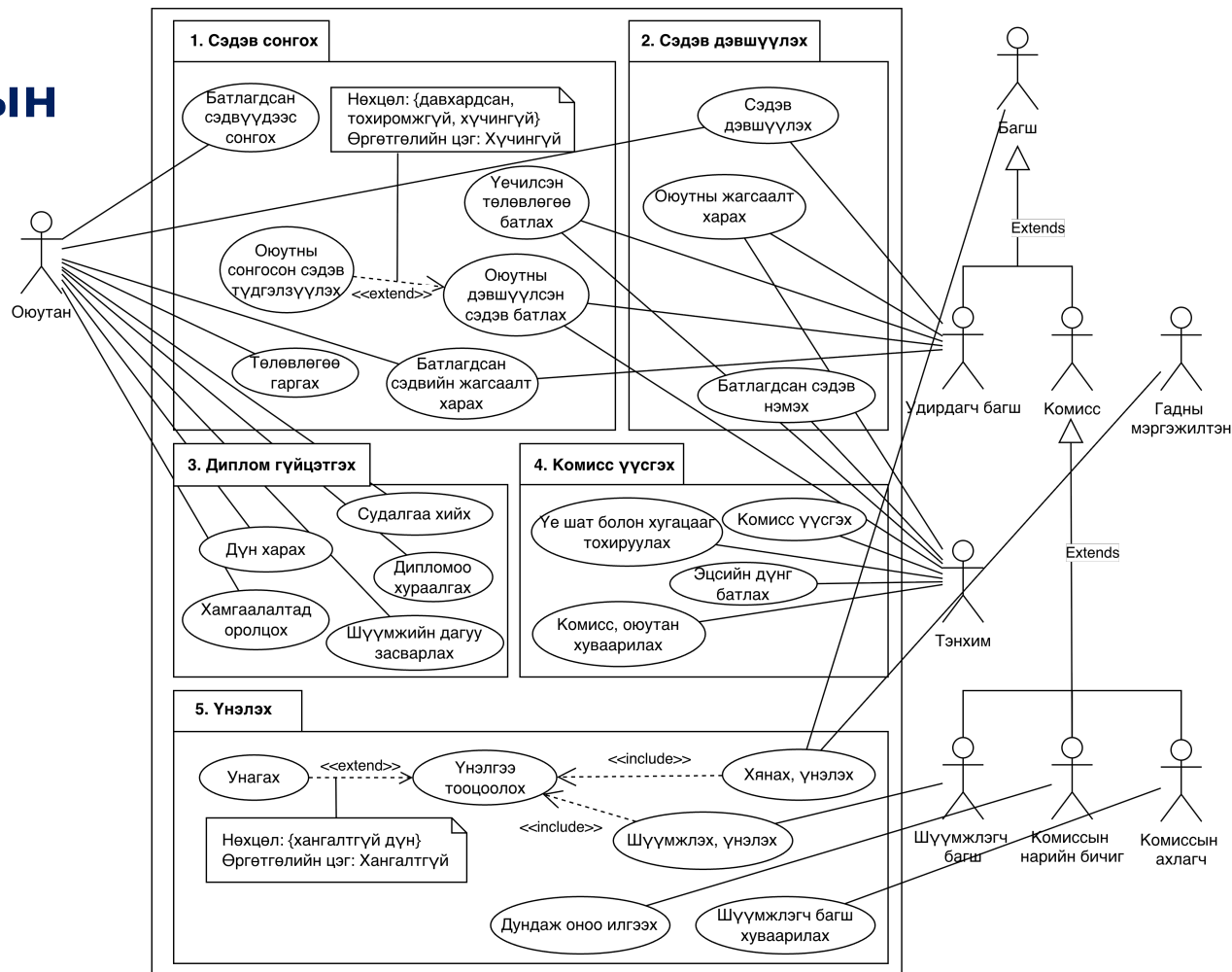
Туршилтын орчны динамик загвар



“Дипломын ажлын удирдлагын систем”-ийн ажлын явц

Оролцогч талууд:

- Оюутан
- Багш
- Тэнхим
- Гадны мэргэжилтэн

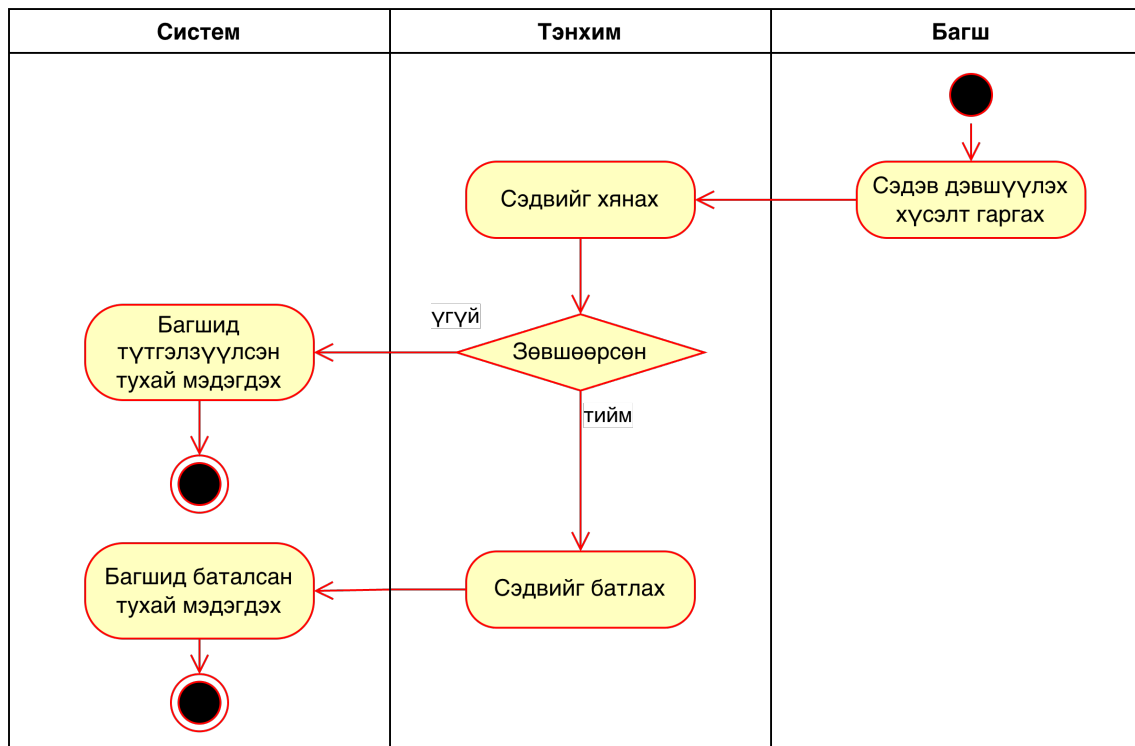


Туршилтын орчны функциональ загвар



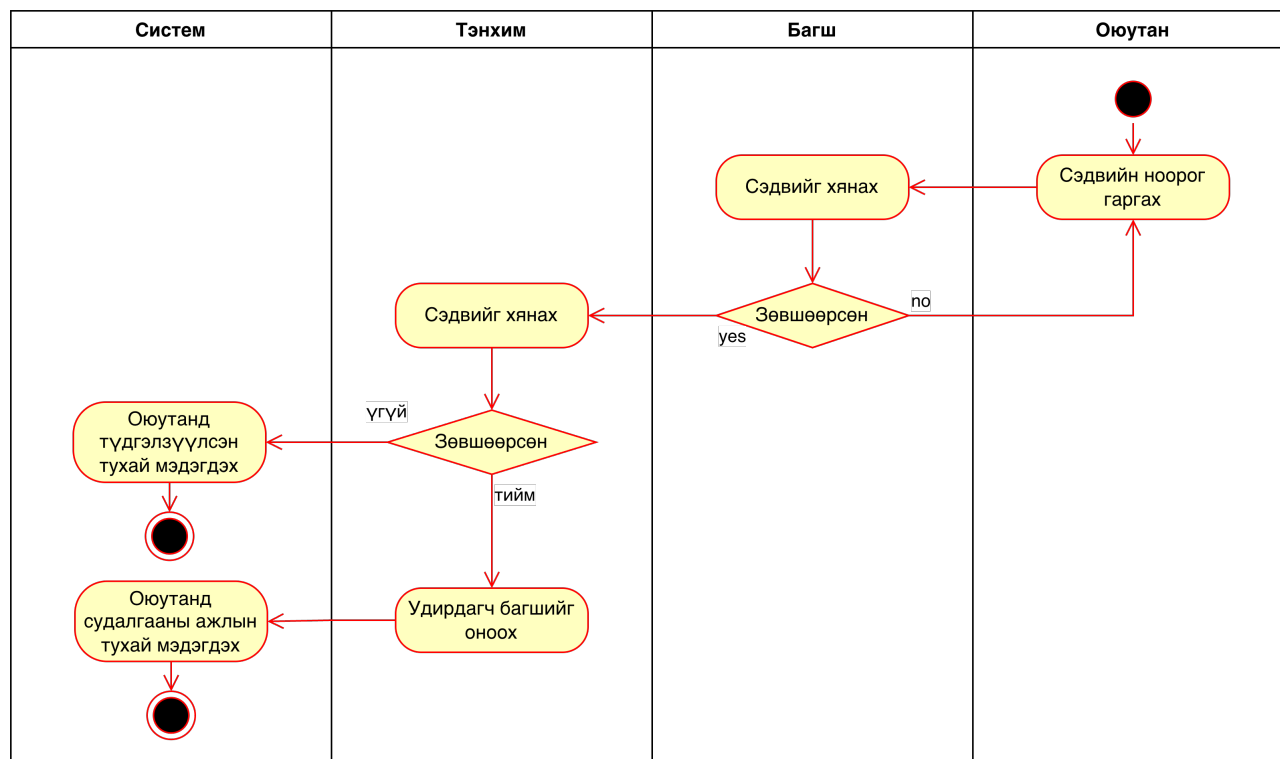
Процесс

БСА-ын сэдэв
дэвшүүлэх



Процесс

БСА-ын сэдэв
дэвшүүлэх –
өргөтгөл:
Оюутан БСА-ын
сэдэв дэвшүүлэх

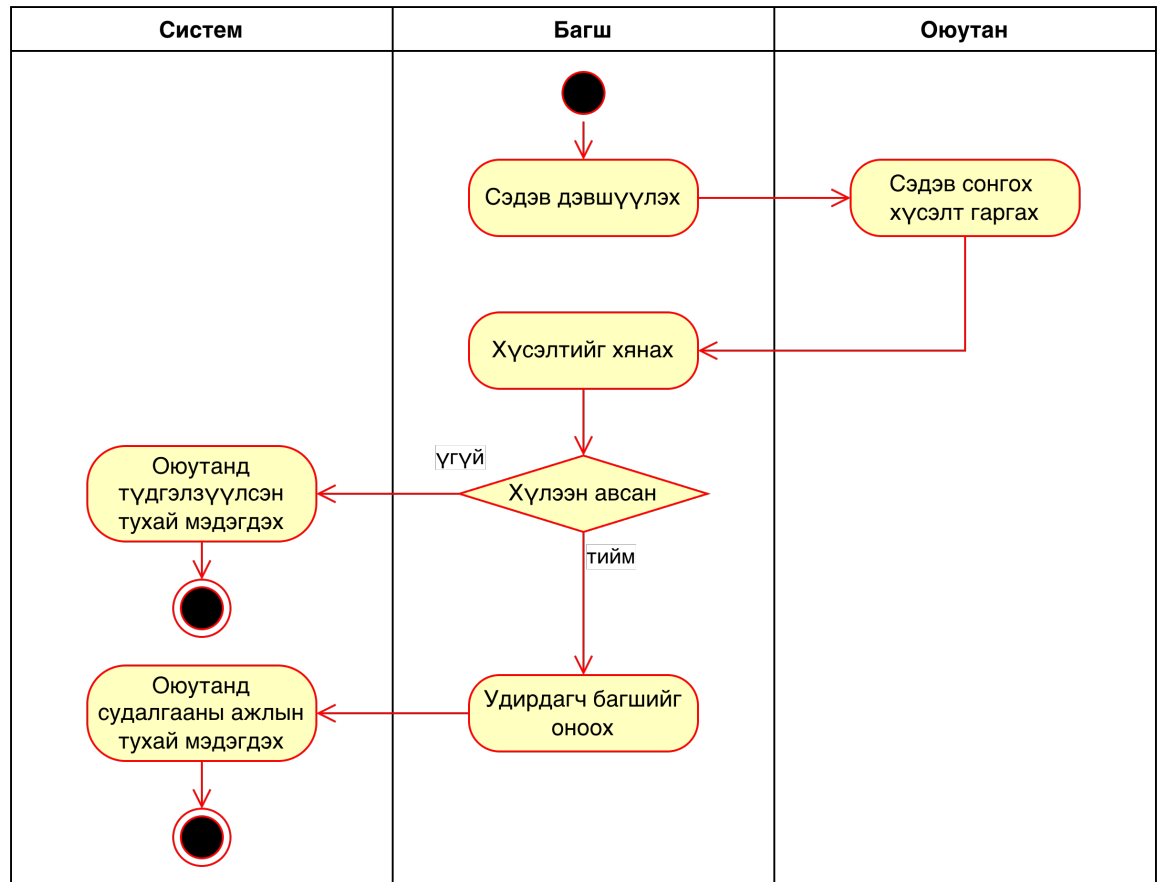


Туршилтын орчны функциональ загвар



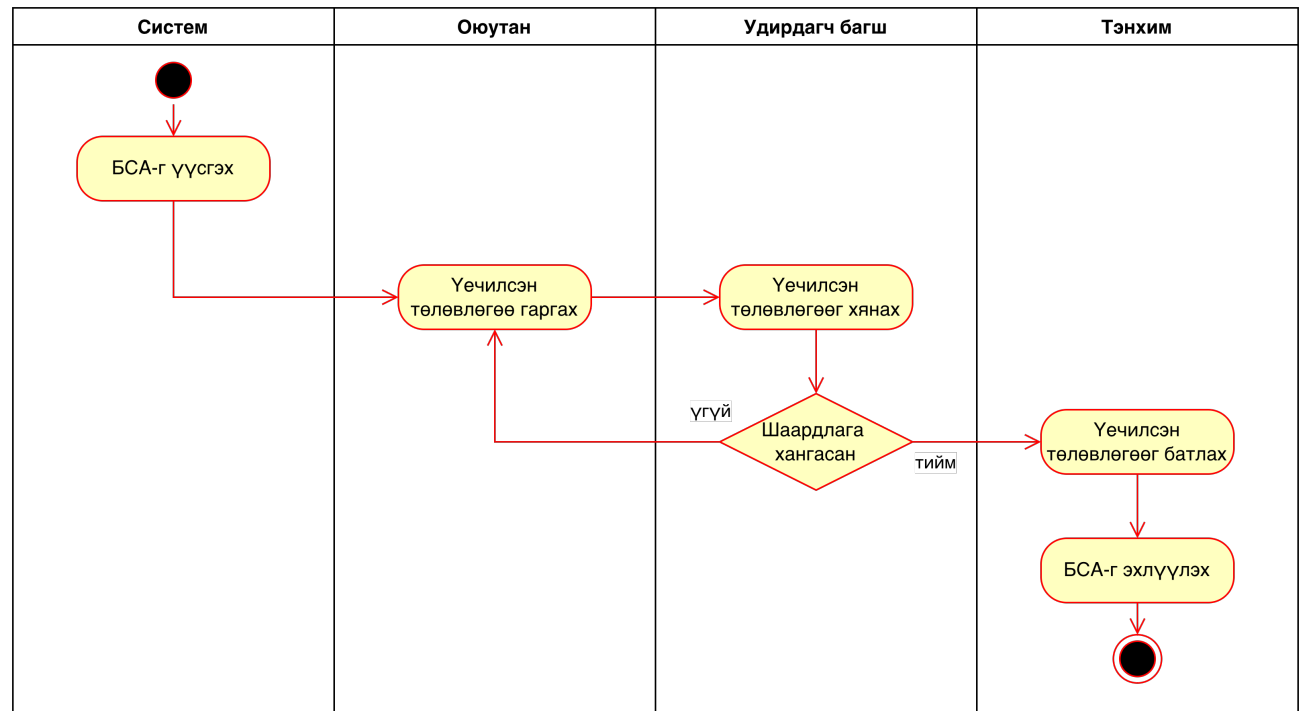
Процесс

БСА-ын сэдэв сонгох



Процесс

БСА-ын үечилсэн
төлөвлөгөө батлах

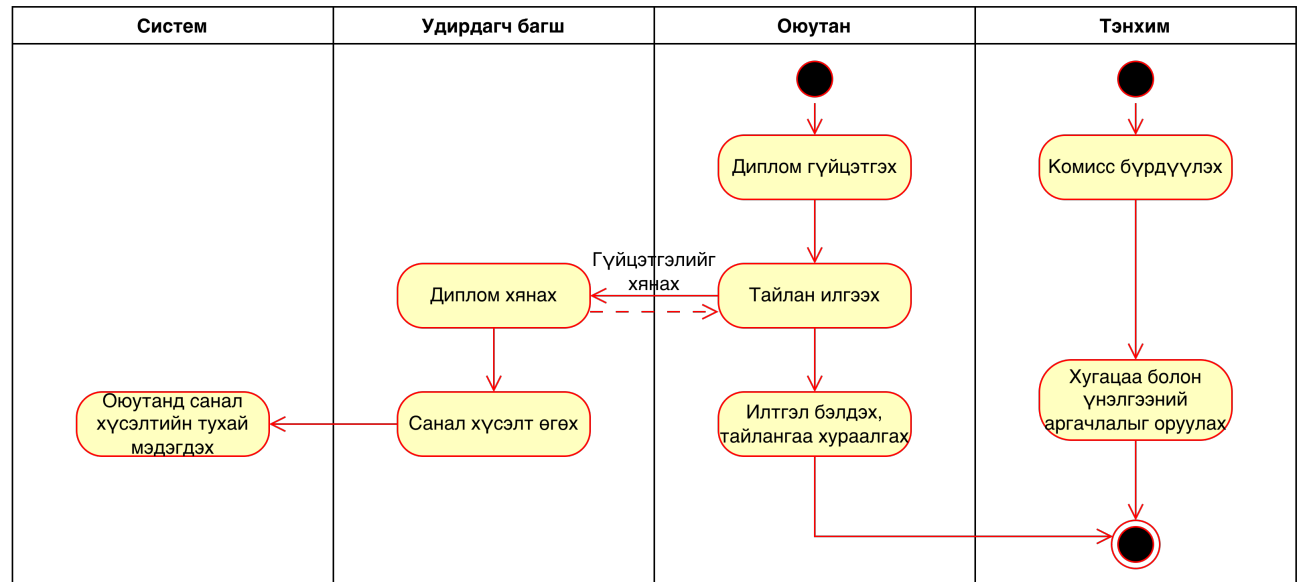


Туршилтын орчны функциональ загвар



Процесс

БСА гүйцэтгэх,
тайлан хураалгах

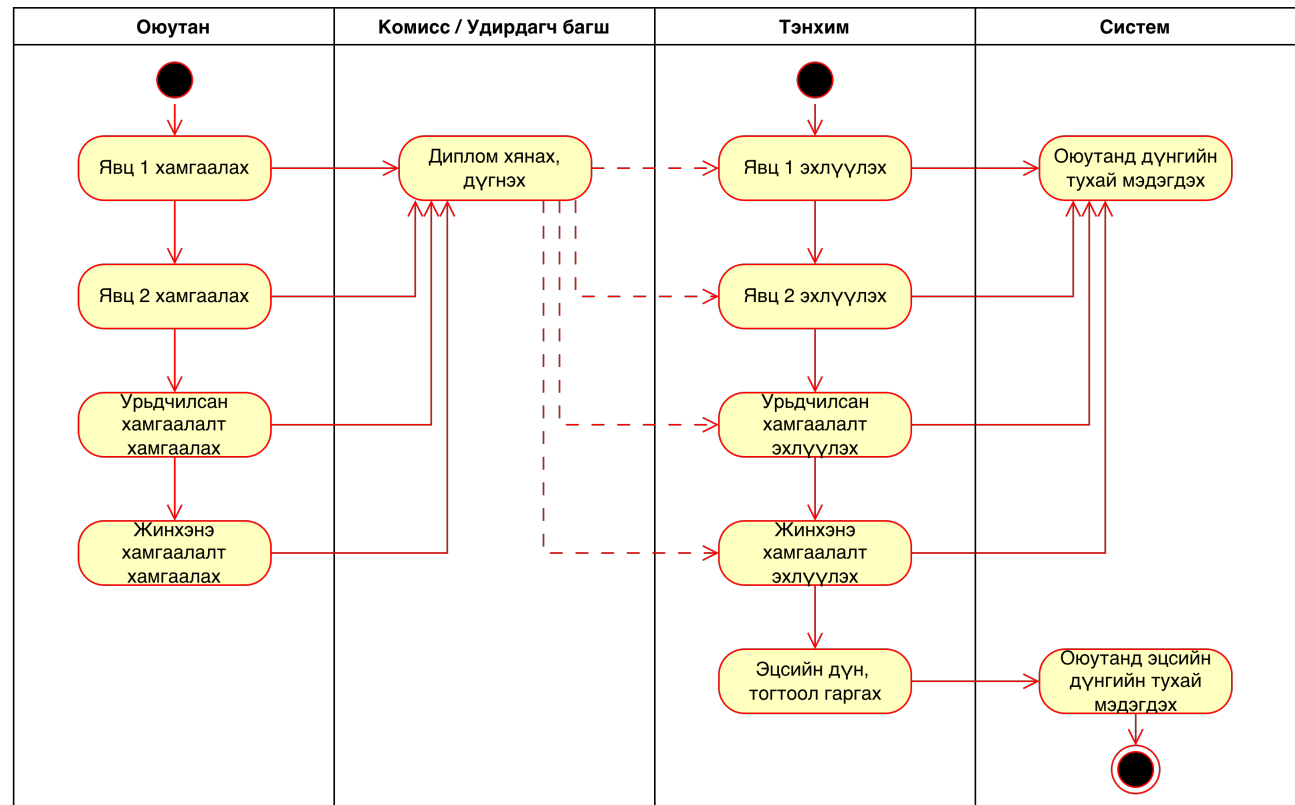


Туршилтын орчны функциональ загвар



Процесс

БСА-ыг хамгаалах,
дүгнэх



Туршилтын орчны UI/UX зохиомж



NUM Thesis Management System Free

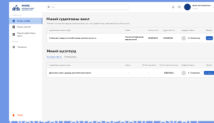


8%



Share

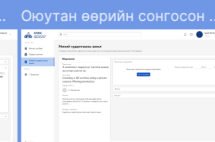
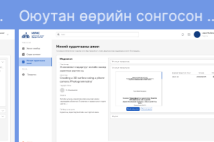
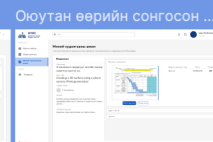
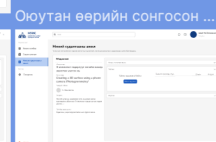
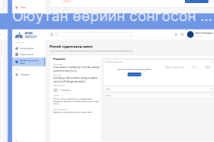
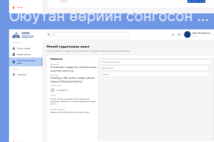
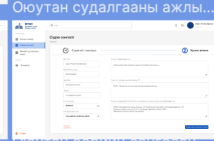
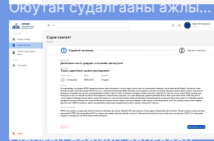
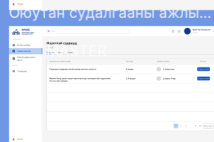
Оюутан
Хянах самбар



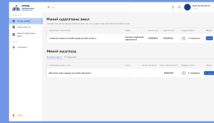
Хянах самбар - Оюутан д...



Хянах самбар 2



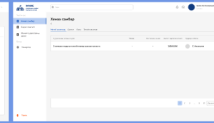
Багш
Хянах самбар



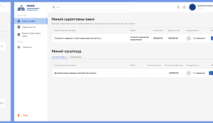
Хянах самбар - Оюутан д...



Хянах самбар 3



Хянах самбар



Хянах самбар - Оюутан д...



Хянах самбар 6



Тэнхим
Оюутан өөрийн сонгосон ...



Оюутан өөрийн сонгосон ...



Оюутан өөрийн сонгосон ...



Оюутан өөрийн сонгосон ...



Оюутан өөрийн сонгосон ...



Оюутан өөрийн сонгосон ...

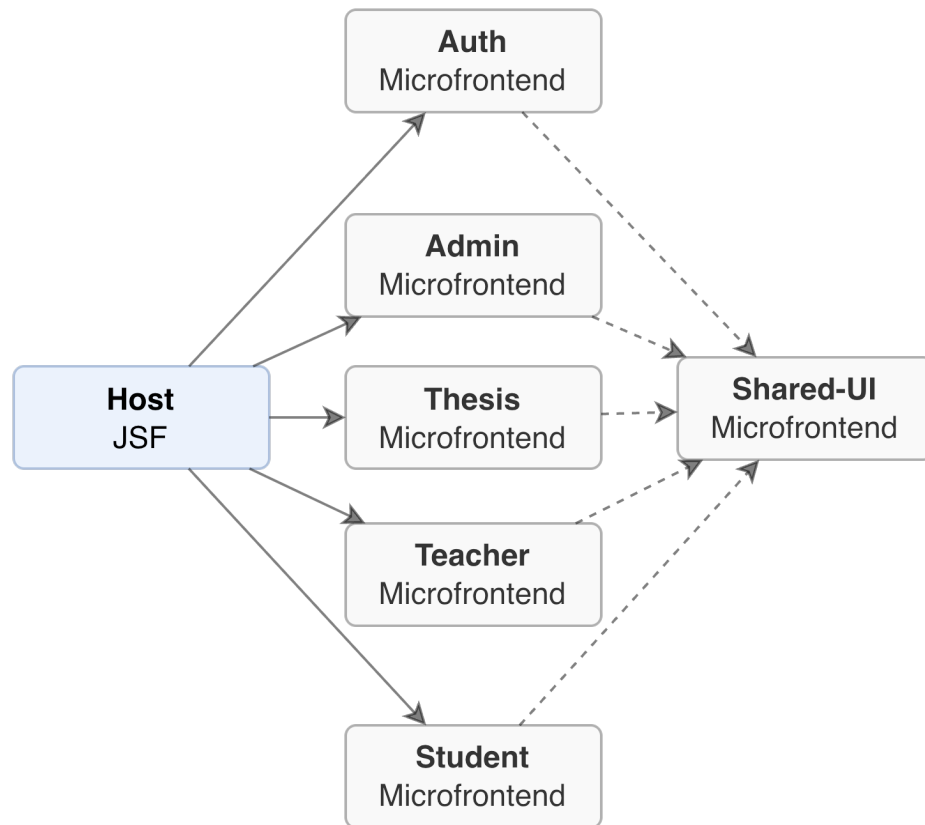


Фронтэнд: Микрофронтэнд архитектур



Бизнесийн хил хязгаараар 7 модульд хуваасан:

- ❖ **Host (JSF):** Үндсэн бүрхүүл, чиглүүлэлт, MFE-үүдийг нэгтгэгч;
- ❖ **Module-Auth:** Төвлөрсөн нэвтрэлт, аюулгүй байдал;
- ❖ **Module-Student/Teacher/Admin:** Хэрэглэгчийн үүрэгт суурилсан бизнесийн логикууд;
- ❖ **Shared UI:** Зохиомжийн нэгдмэл байдлыг хангах дундын нөөц.



Ажиллах үеийн интеграци нь remoteEntry.js ашиглан lazy-loading зарчмаар ажиллана.

Бэкенд: Микросервис архитектур



Бизнесийн домэйнээр 11 бие
даасан сервист хуваасан:

- **Core Services:**

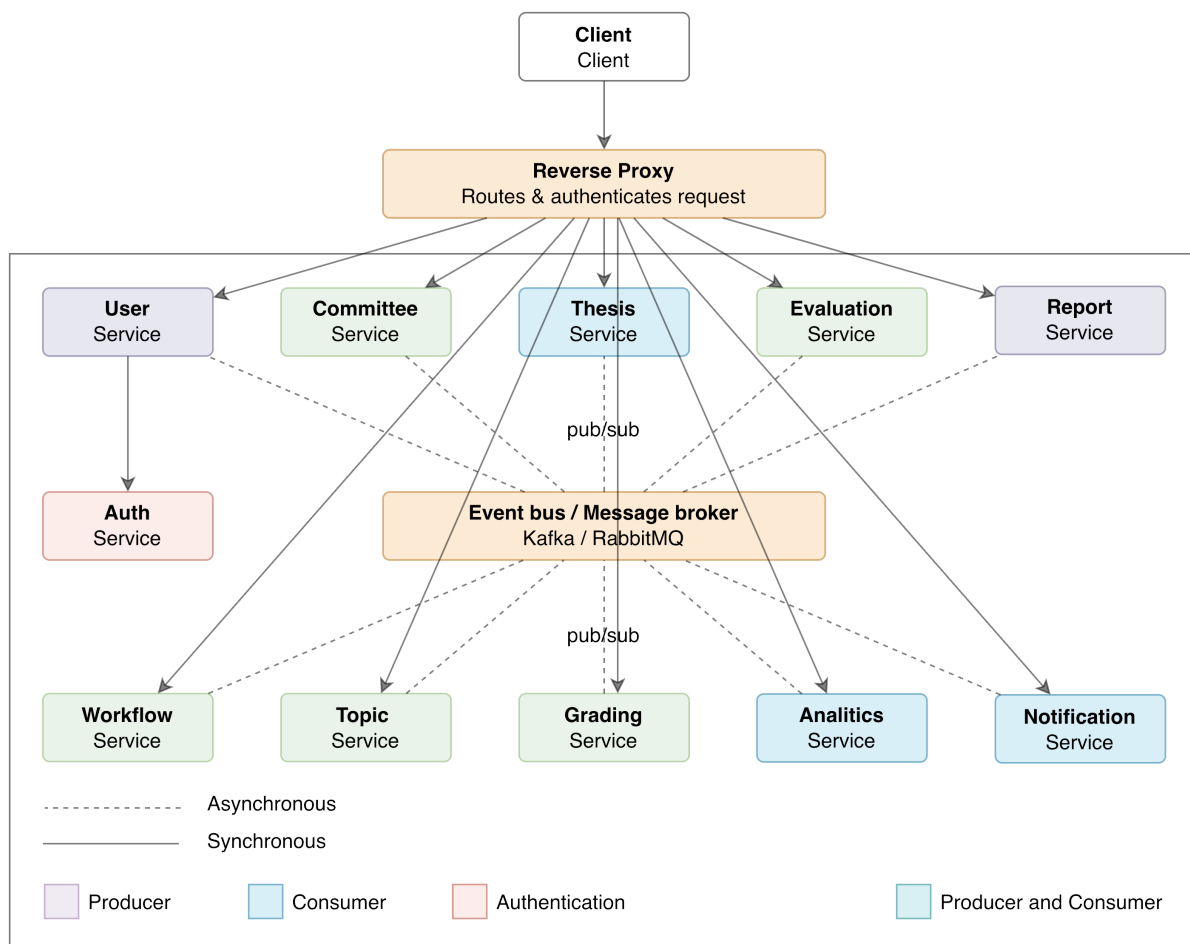
- ❖ thesis-service
- ❖ topic-service
- ❖ user-service
- ❖ committee-service
- ❖ evaluation-service
- ❖ workflow-service
- ❖ grading-service

- **Support Services:**

- ❖ notification-service
- ❖ report-service
- ❖ analytic-service

- **Infrastructure:**

- ❖ auth-service

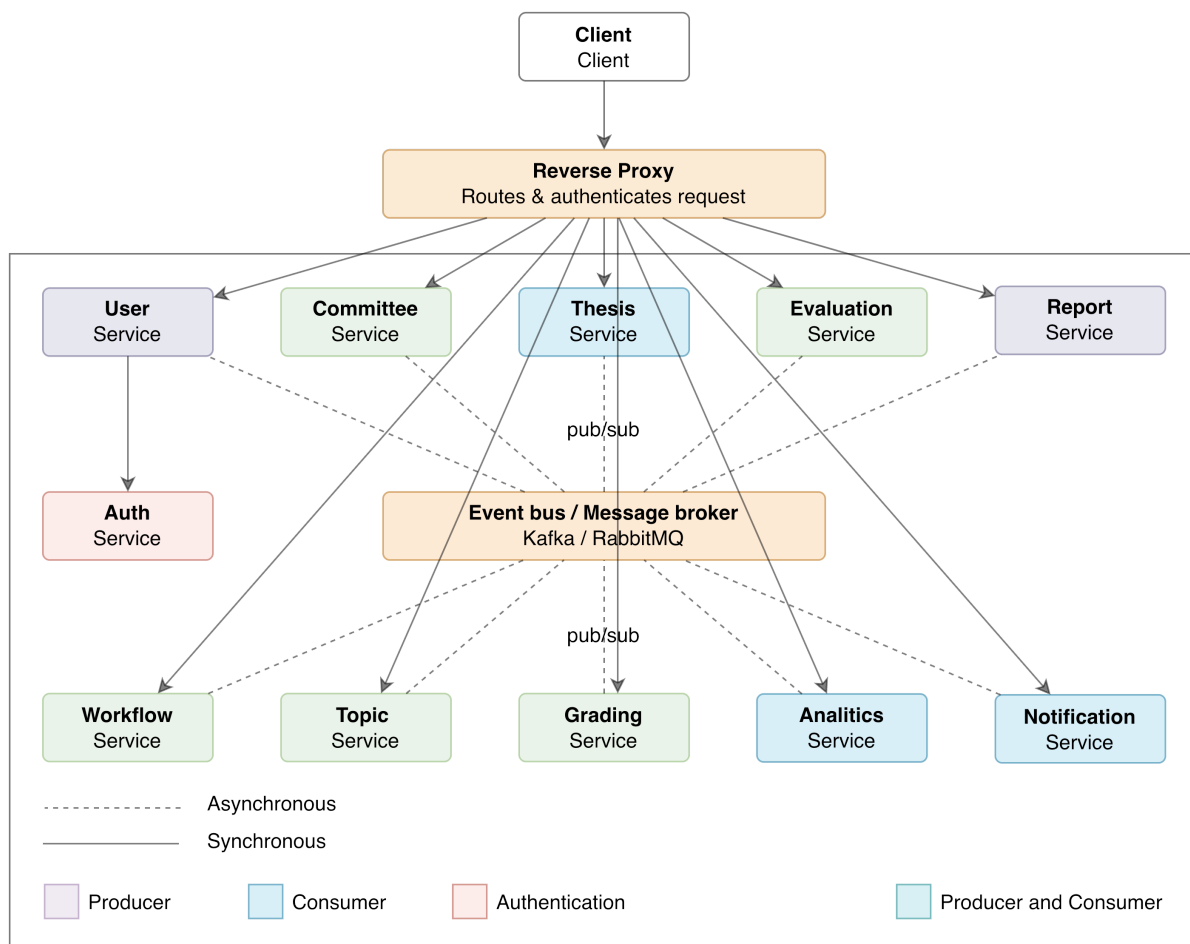


Бэкенд: Микросервис архитектур



Сервисүүд өөр
хоорондоо мэдээлэл
солилцох нь:

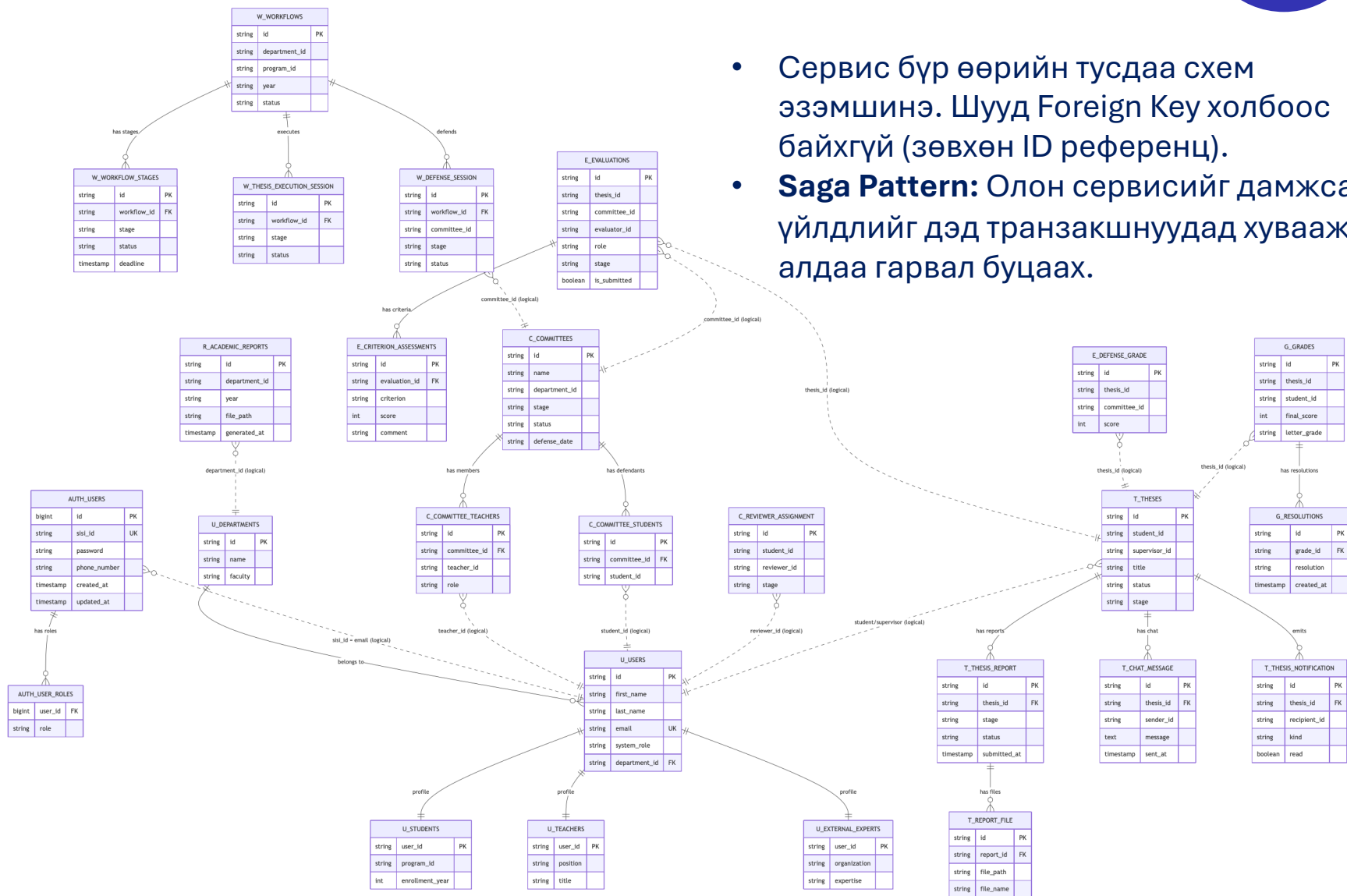
- **Синхрон (REST/HTTP):** Бодит хугацааны хариу шаардлагатай хүсэлтүүд
- **Асинхрон (Event-driven):** Мэдэгдэл илгээх, аудит хийх зэрэгт Message Broker ашиглана.
- **API-First хандлага:** Давхарга хоорондын харилцаа зөвхөн баримтжуулсан API гэрээн дээр үндэслэгдэнэ.



Өгөгдлийн давхарга: Логик тусгаарлалт



- Сервис бүр өөрийн тусдаа схем эзэмшинэ. Шууд Foreign Key холбоос байхгүй (зөвхөн ID референц).
- Saga Pattern:** Олон сервисийг дамжсан үйлдлийг дэд транзакшнуудад хувааж, алдаа гарвал буцаах.



GitHub push (main)

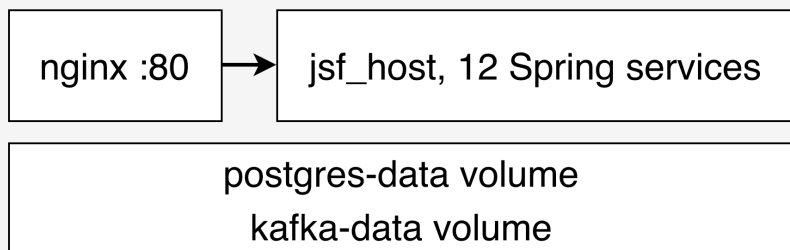


CI/CD дамжлага (.github/workflows/02-deploy):

- | | |
|-----------------|--------------------------------|
| 1. paths-filter | өөрчлөгдсөн сервисийг илрүүлэх |
| 2. setup-server | scp compose, nginx.conf |
| 3. buildx | Docker Hub (:sha, :latest) |
| 4. ssh-action | docker compose pull, up |



Сервер (Linux, Docker 24+):



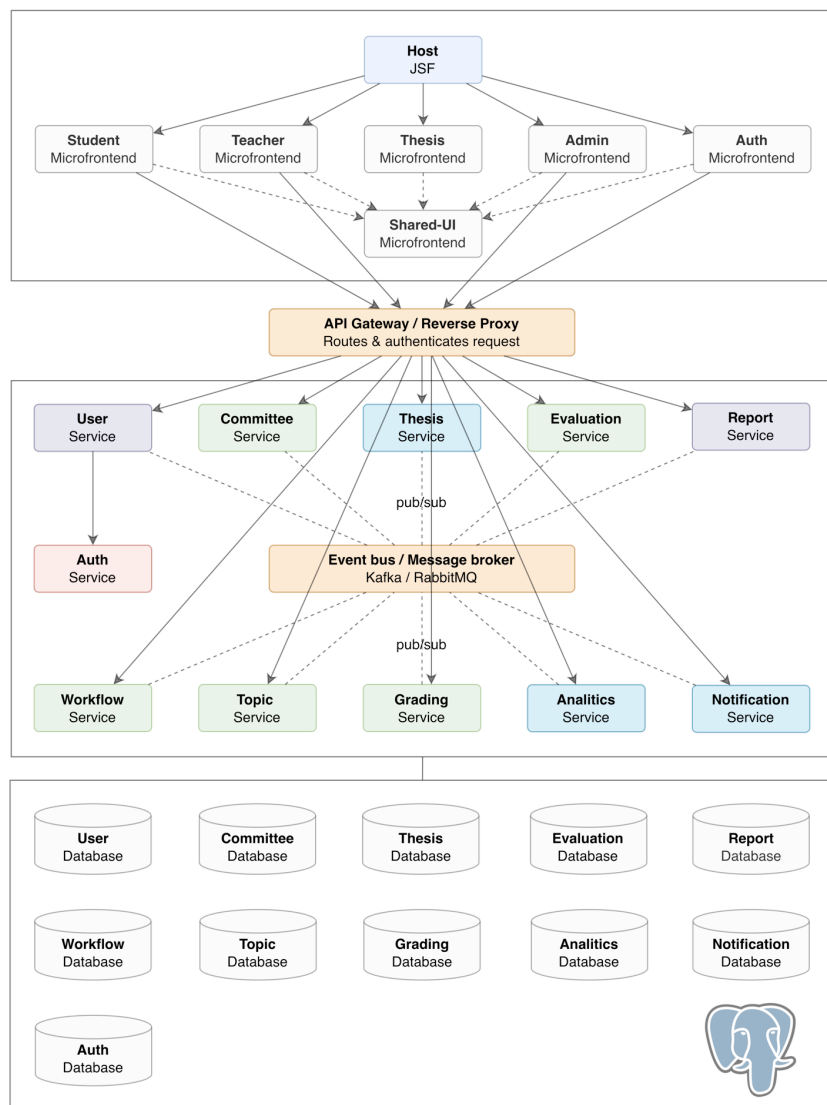
15 контейнер: nginx (1), JSF host (1), auth (1), message (1), Spring service (10), Kafka (1).

- **--no-deps** (per-service deploy)
- **:<sha>** (rollback handle)
- **:latest** (active head pointer)

docker-compose.prod.yml:

Docker Hub-аас pre-built image татаж, GitHub Actions SSH-ээр асаана.

Туршилтын орчны тойм

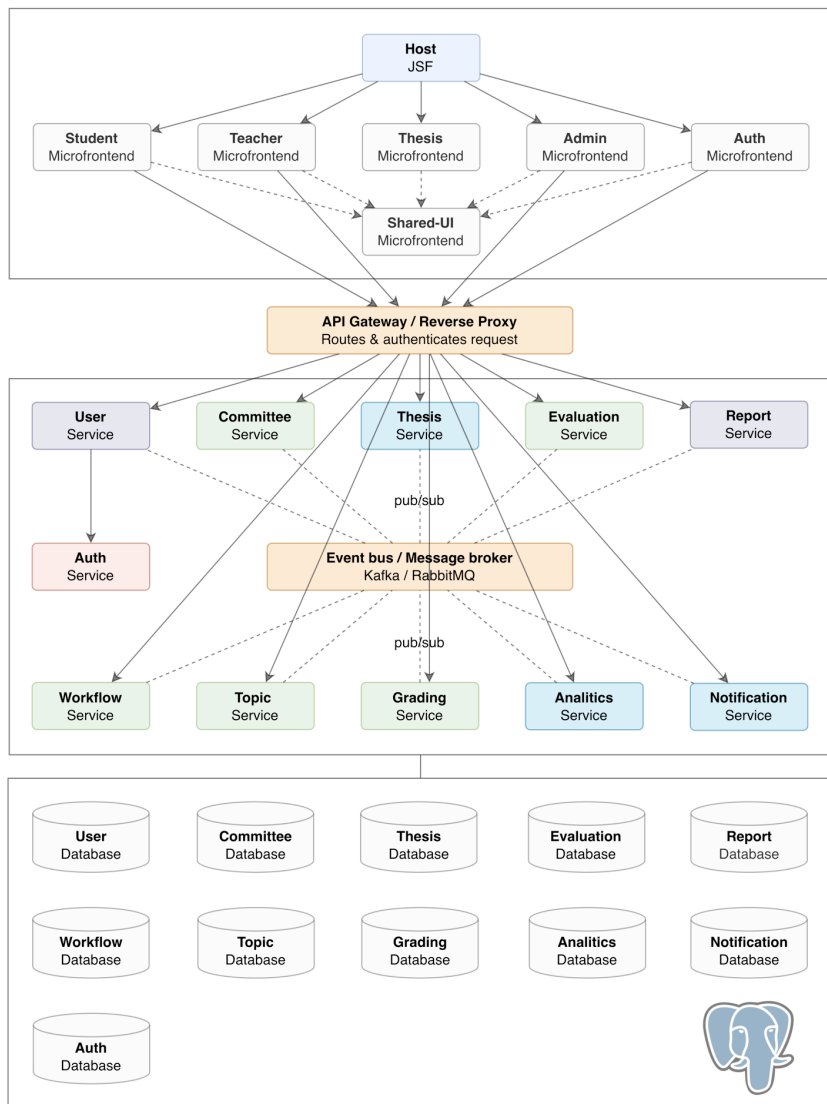


Ашигласан технологи:

- **Frontend:** JSF Host + Vite Module Federation (React).
- **Backend:** Java/Spring WebFlux микросервисүүд.
- **Database:** PostgreSQL/R2DBC (Schema-per-service).

Техник хангамж: Apple M1 Pro, 16GB RAM (JDK 17, JDK 21, Node.js 20).

Туршилтын орчны тойм



Хэмжилтийн 4 тулгуур:

- **Хөгжүүлэлтийн гүйцэтгэл:** Build time, HMR хурд.
- **Ачааллын гүйцэтгэл:** RAM ашиглалт, HTTP латенц.
- **Сүлжээний зардал:** Payload хэмжээ, HTTP кэшийн эффект.
- **Кодын чанар:** SonarQube шинжилгээ, Cyclomatic complexity.

Хэрэгслүүд: Apache JMeter, SonarQube, Jaeger (Tracing), Hyperfine.

Хөгжүүлэлтийн бүтээмж (Build & HMR)

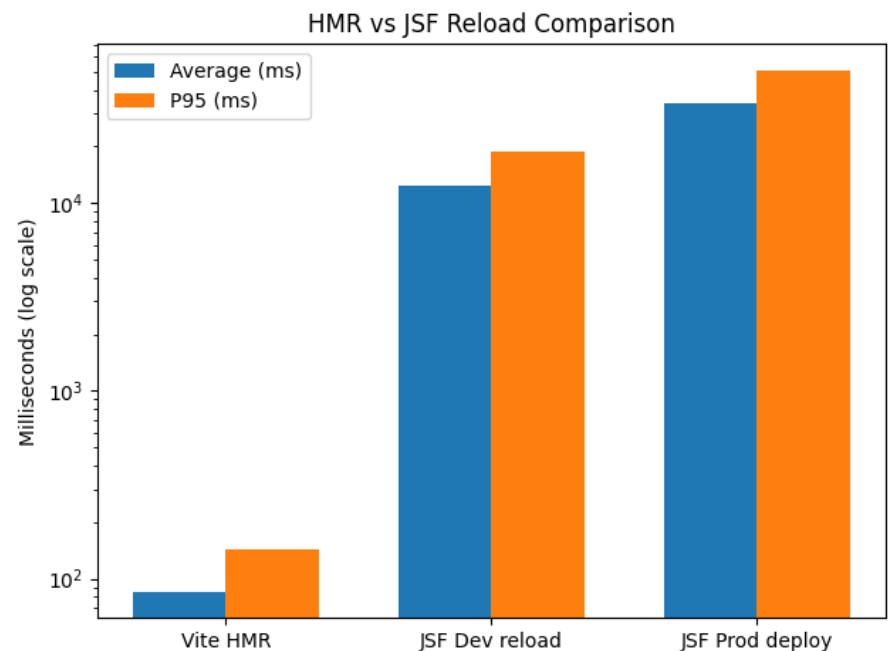


Build Pipeline: JSF (70.5c) vs React MFE (12.7c). Үр дүн: 5.5 дахин хурдан.

HMR (Hot Module Replacement):

- **Vite HMR:** 85 мс
- **JSF Reload:** 12,400 мс.
- **Дүгнэлт:** JSF-ийн 12 секундын хүлээлт нь хөгжүүлэгчийн "Flow" буюу төвлөрлийг тасалдуулж, бүтээмжийг 145 дахин бууруулж байна.

Lead Time: MFE-ийн анхны суурь тохиргоо (TypeScript, Contracts) цаг их авч байгаа ч чанарын хяналтыг (QA) сайжруулсан.



Системийн ачаалал даах чадвар (RAM & GC)

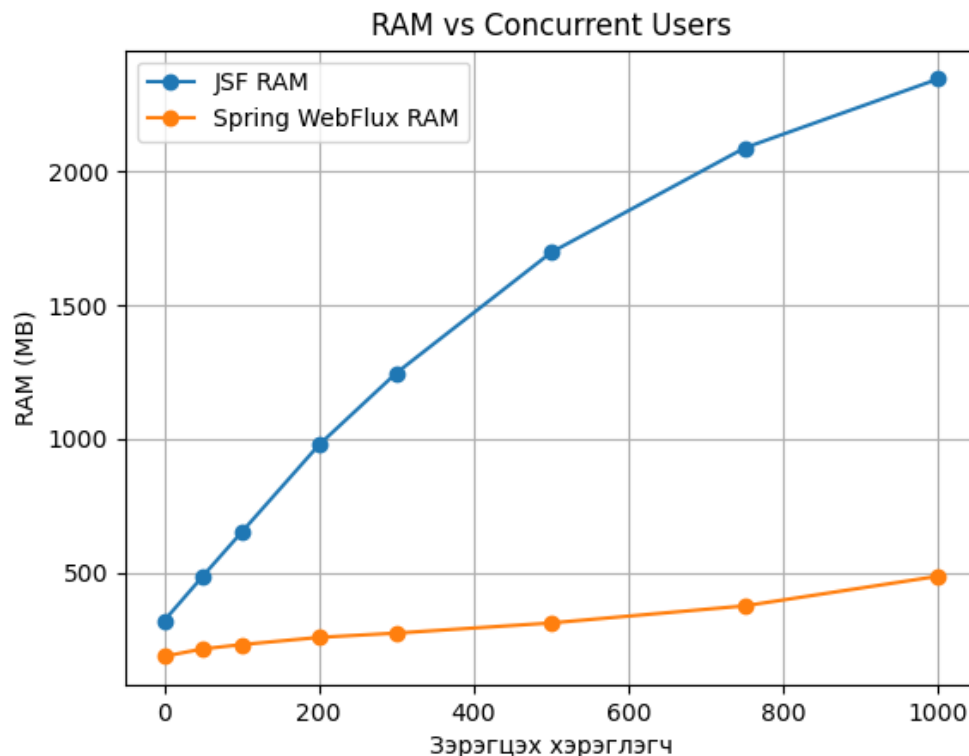


RAM ашиглалт (1,000 хэрэглэгч зэрэг хандахад):

- **JSF (Monolith):** 2,347 МБ.
- **Spring WebFlux:** 487 МБ.

Ялгаа: 4.8 дахин бага нөөц ашиглалт.

Шалтгаан: JSF-ийн HttpSession объект бүр RAM-д ачаалал өгдөг бол WebFlux-ийн Event-loop нь I/O thread-үүдийг оновчтой хуваарилдаг (Non-blocking).



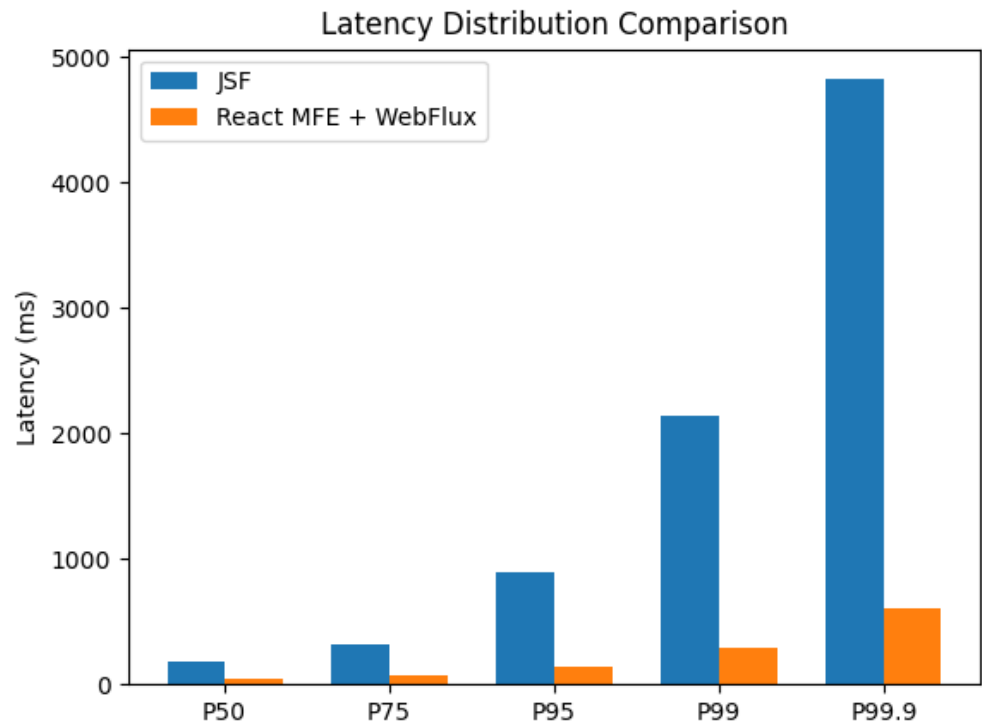
Хариу өгөх хугацаа (HTTP Latency)



P99 Latency (1,000 хэрэглэгч дээр):

- **JSF:** 2,147 мс (SLA хязгаараас давсан).
- **MFE + WebFlux:** 287 мс (Маш хурдан).

Дүгнэлт: Шинэ архитектур хариу өгөх хугацааг 7.4 дахин хурдасгасан. Энэ нь систем ачааллын үед гацахгүй, тогтвортой ажиллах баталгаа юм.

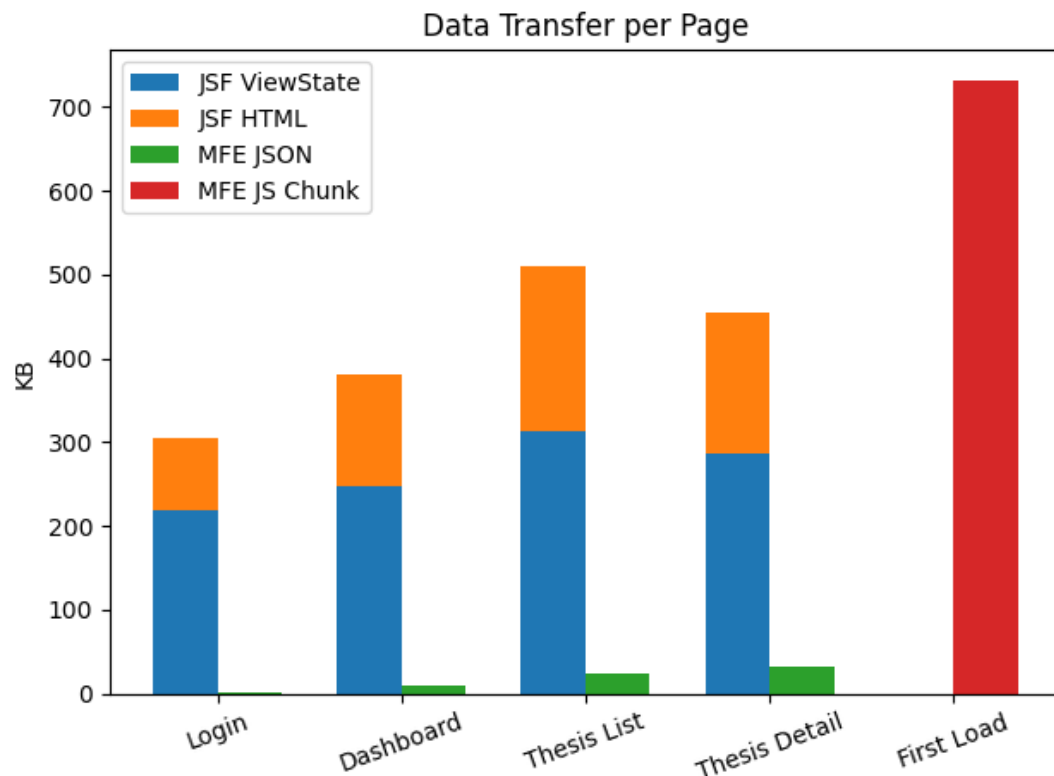


Анхны ачаалал: MFE нь 731 КБ (Antd/React) татаж байгаа ч энэ нь 1 жилийн хугацаатай кэшлэгдэнэ.

Хуудас шилжилт:

- **JSF:** Хуудас бүрт 200-300 КБ (ViewState + HTML).
- **MFE:** Зөвхөн 8-30 КБ (JSON дата).

Эдийн засгийн үр ашиг: 8-аас дээш хуудас шилжихэд MFE нь JSF-ээс илүү хэмнэлттэй болж эхэлнэ. 100 шилжилт хийхэд өгөгдөл дамжуулалтыг 9 дахин хэмнэнэ.



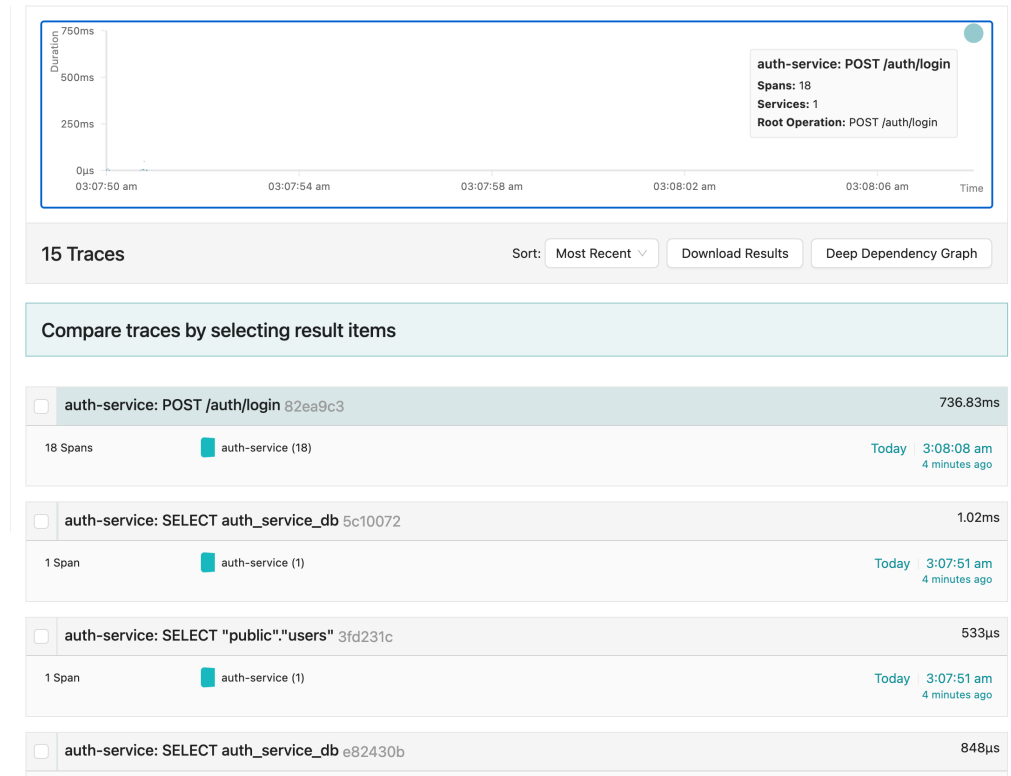
SonarQube үзүүлэлт:

- **Code Smells:** 47 (JSF) → 23 (MFE).
- **Цикломатик нарийн төвөгтэй байдал:** 12.3 → 6.8 (44% буурсан).

Алдаа оношлох хугацаа (MTTD):

- **JSF (Лог файл хайх):** ~23 минут.
- **MFE (Jaeger Tracing):** 30 секунд.

Дүгнэлт: Логик илүү ойлгомжтой болж, алдааг илрүүлэх хугацаа бодитоор буурсан.



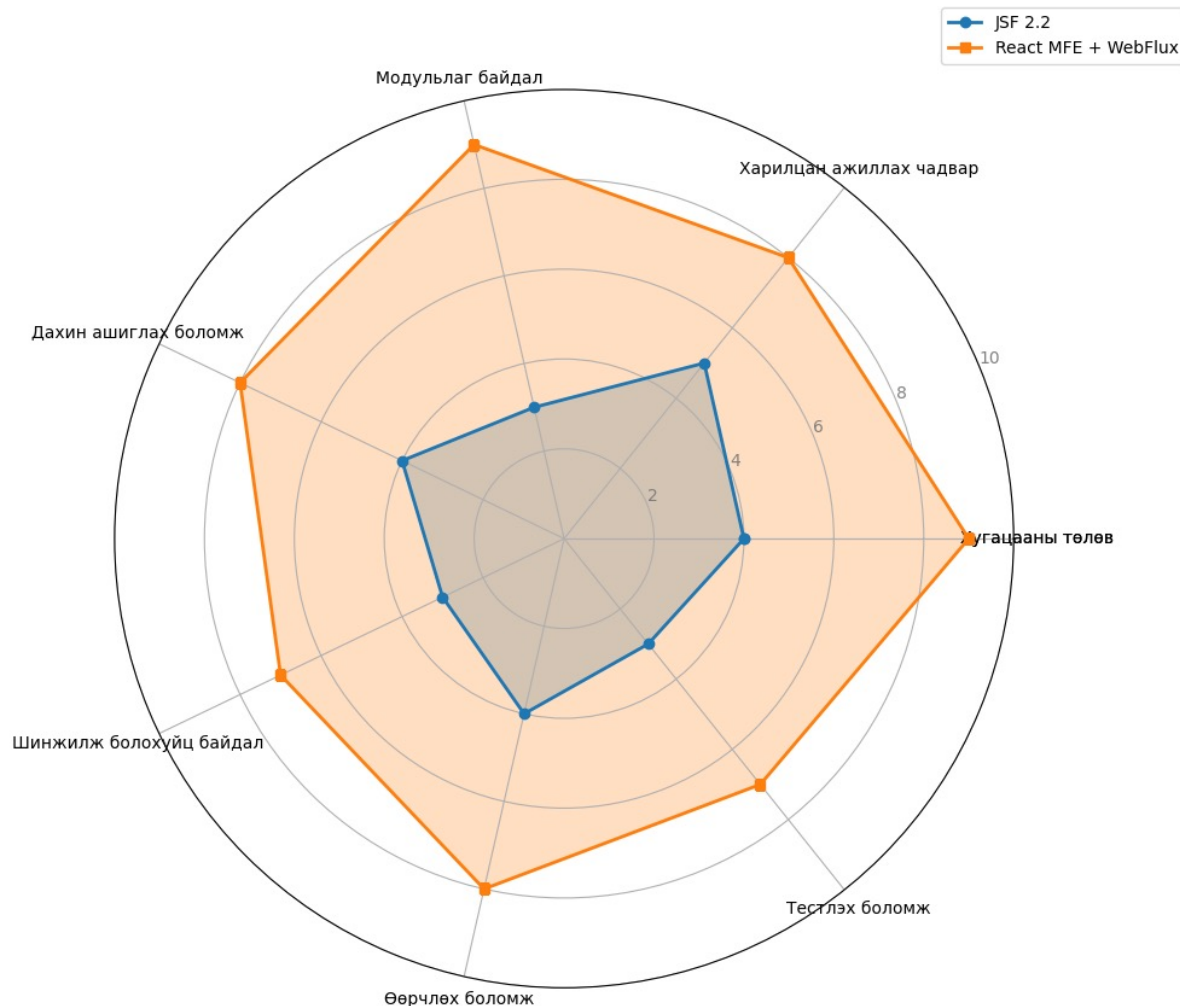
Чанарын өсөлт: ISO

25010 стандартын 7 үзүүлэлтээр дундаж оноо 3.71-ээс 8.00 болж өссөн (115.6%-ийн чанарын ахиц).

Гол давуу тал: Модульлаг байдал болон Гүйцэтгэл.

Архитектурын "Trade-off": Реактив систем нь хурд өгөх боловч алдаа оношлоход нарийн төвөгтэй (үүнийг Jaeger-ээр шийдсэн).

ISO/IEC 25010 Чанарын үзүүлэлтийн харьцуулалт



- 1 Удиртгал
- 2 Зорилго, Зорилт
- 3 Судалгааны арга зүй
- 4 Туршилт, Үр дүн
- 5 **Дүгнэлт**

Энэхүү бакалаврын судалгааны ажлын хүрээнд МУИС-ийн Дипломын ажлын удирдлагын системийн хэрэглэгчийн интерфэйс дээр React болон JSF технологийг хослуулсан холимог микрофронтэнд архитектурыг туршиж, ISO/IEC 25010 стандарт болон DORA, ATAM аргачлалаар үнэлж дүгнэлээ. Дэвшүүлсэн гурван зорилт бүрэн биелэгдэв:

- **Судалгаа хийх** зорилтоор микрофронтэнд архитектурын онол, арга зүй болон ISO/IEC 25010 стандартыг судалж, холбогдох 10 эрдэм шинжилгээний бүтээлийн тойм хийв.
- **Туршилтын загвар боловсруулах** зорилтоор Vite Module Federation ашиглан JSF хост болон React MFE модулиудыг нэгтгэсэн туршилтын загварыг боловсруулж, уг туршилтын загварын дагуу МУИС-ын Дипломын ажлын удирдлагын системийг хэрэгжүүлэв.
- **Туршилт ба үнэлгээ хийх** зорилтоор ISO/IEC 25010 стандартын 7 чанарын үзүүлэлтийг Google Lighthouse, SonarQube, madge, Jest, Cypress хэрэгслүүдээр хэмжиж, JSF болон React MFE архитектурыг харьцуулан шинжилж, ATAM аргачлалаар харьцуулалт хийв.

Анхаарал хандуулсанд баярлалаа!

Ном зүй - Хураангуй

- [1] International Organization for Standardization. ISO/IEC 25010:2011 Systems and software Quality Requirements and Evaluation (SQuaRE). Mar 2019. URL: <https://www.iso.org/standard/35733.html>
- [2] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina. Microservices: Yesterday, Today, and Tomorrow. Cham: Springer International Publishing, 2017, pp. 195–216. URL: https://doi.org/10.1007/978-3-319-67425-4_12
- [3] Atlassian. Microservices architecture. URL: <https://www.atlassian.com/microservices/microservices-architecture>
- [4] Kubernetes. URL: <https://kubernetes.io/>
- [5] M. Mikowski and J. Powell. Single Page Web Applications: JavaScript end-to-end. Manning, 2013. URL: <https://books.google.se/books?id=eDszEAAAQBAJ>
- [6] Micro frontends. URL: <https://martinfowler.com/articles/micro-frontends.html>
- [7] M. Fowler. Strangler Fig Application. martinfowler.com, Jun 2004. URL: <https://martinfowler.com/bliki/StranglerFigApplication.html>

Нөөц үзүүлэн

Дипломын ажлын удирдлагын систем



Гарын авлага

